Open in app ↗

Medium Search Write A

Get unlimited access to the best of Medium for less than $1/week. Become a member

Expedia Group Technology

EXPEDIA GROUP TECHNOLOGY — DATA

Apache Spark Structured Streaming — Watermarking (6 of 6)

Watermarking in Spark Streaming to handle late data



Neeraj Bhadani · Follow

Published in Expedia Group Technology · 5 min read · Mar 11, 2021

104

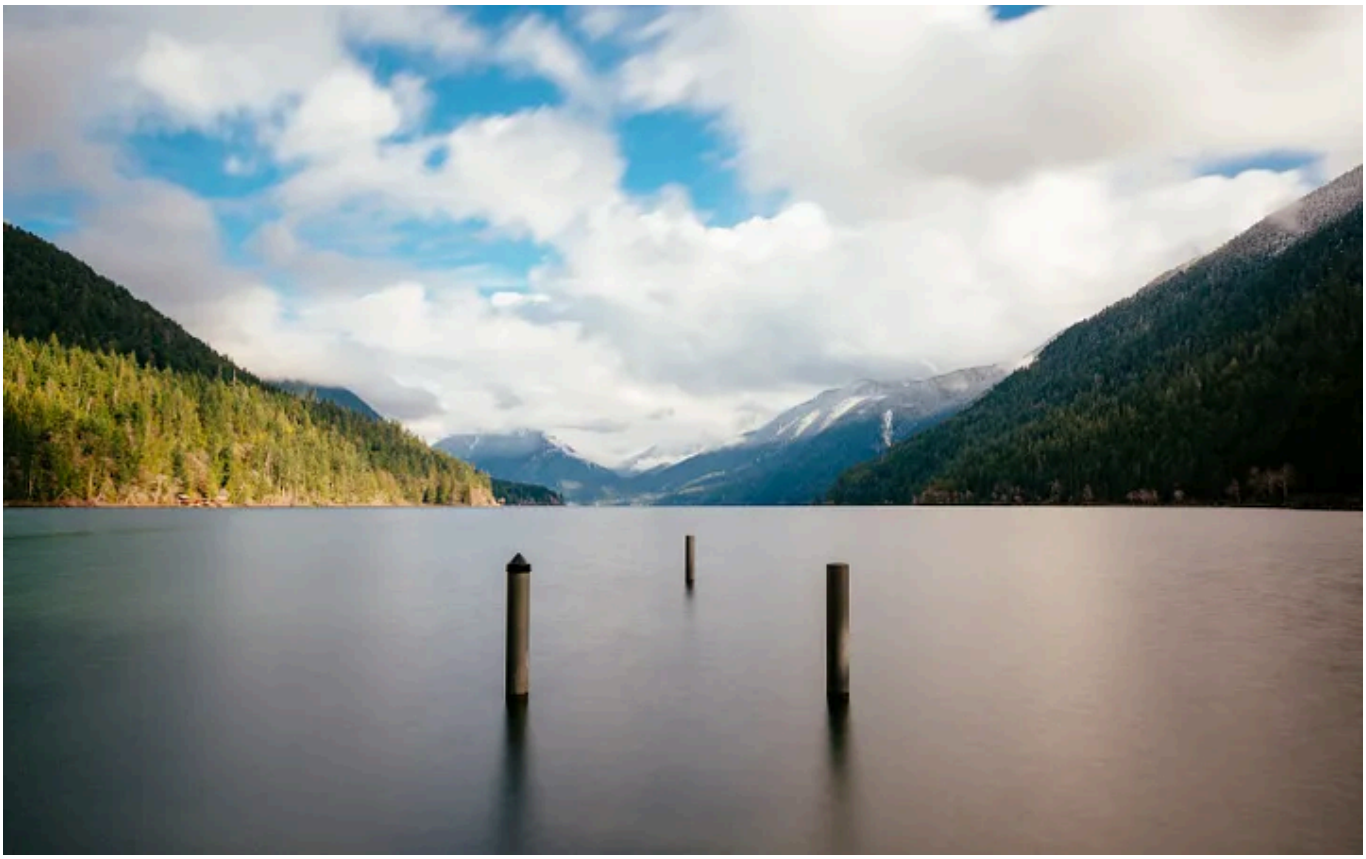


Photo on unsplash.com by Dave Hoefler

In previous blogs, we discussed input sources, sinks, checkpoints, triggers and operations. In this post, we discuss watermarking in Apache Spark™ Streaming.

You may also be interested in some of my other posts on Apache Spark.

- Apache Spark Structured Streaming — First Streaming Example
- Apache Spark Structured Streaming — Input Sources
- Apache Spark Structured Streaming — Output Sinks
- Apache Spark Structured Streaming — Checkpoints and Triggers
- Apache Spark Structured Streaming — Operations
- Deep Dive into Apache Spark DateTime Functions

- Working with JSON in Apache Spark
- Deep Dive into Apache Spark Window Functions
- Deep Dive into Apache Spark Array Functions
- Start Your Journey with Apache Spark

Let's set-up a streaming application by reading data from a socket to understand the need and concept of watermarking with an example.

- - - - - -

Setup

We read data from a socket (`127.0.0.1:9999`) and pass along data which is `#` separated. Then we create a DataFrame with 2 columns `event_timestamp` and `val` . More details on reading data from a socket can be found in this blog.

```scala
// Import libraries
import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.functions._

// Create Spark Session
val spark = SparkSession
  .builder()
  .master("local")
  .appName("WaterMark")
  .getOrCreate()

// Set Spark logging level to ERROR.
spark.sparkContext.setLogLevel("ERROR")

// Define host and port number to Listen.
val host = "127.0.0.1"
val port = "9999"

// Create Streaming DataFrame by reading data from socket.
val initDF = spark
  .readStream
  .format("socket")
  .option("host", host)
  .option("port", port)
  .load()

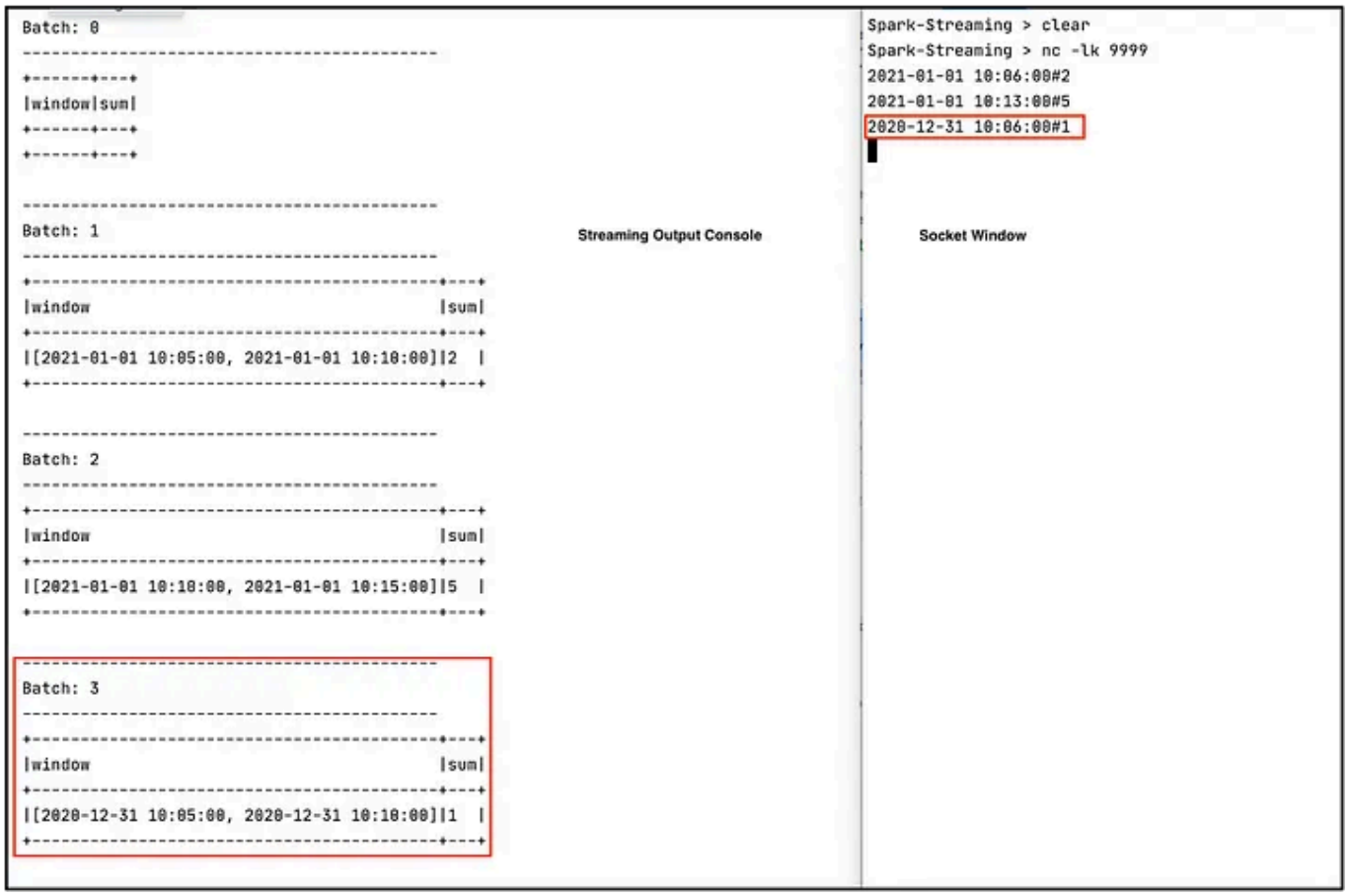
// Create DataFrame with event_timestamp and val column
val eventDF = initDF.select(split(col("value"), "#").as("data"))
  .withColumn("event_timestamp",    element_at(col("data"),1).cast("timestamp"))
  .withColumn("val", element_at(col("data"),2).cast("int"))
  .drop("data")
```

ss_waterMark_setup.scala hosted with ❤ by GitHub view raw

Now let's open port `9999` and send some data, do the window aggregation and display the results on console. We kept the size of the tumbling window as `5 minutes`. More details on `window` operations can be found in this blog.

```scala
val resultDF = eventDF
  .groupBy(window(col("event_timestamp"), "5 minute"))
  .agg(sum("val").as("sum"))
```

ss_waterMark_need.scala hosted with ❤ by GitHub view raw



On right window, we can see input on socket window. On left window, we can see output on console.

The first 2 records are from `1st Jan 2021` while the 3rd record is from the `31st Dec 2020`. Let's consider the third record *late data*. (Please be aware that the definition of late data varies from application to application). With window operations, a streaming application will process all data and put late data into an older window. At the same time, it maintains state for the old data which might not be required anymore. We can use `watermarking` to handle late data so that the system discards records automatically.

Now let's add `withWatermark()` to our application and see how it works. We set `watermark` to `delayThreshold = 10 minutes` as well as `windowDuration = 5 minutes` using `groupBy()`. Watermark is set to `max event time seen so far – delayThreshold`. Let's understand with an example.

```
1  val resultDF = eventDF
2    .withWatermark("event_timestamp", "10 minutes")
3    .groupBy(window(col("event_timestamp"), "5 minute"))
4    .agg(sum("val").as("sum"))
```

ss_waterMark.scala hosted with ❤ by GitHub view raw

Open the port `9999`, start our streaming application and send the same data again to the socket. Sample data can be found here. Let's discuss each record in detail.

1. **First record :** `2021-01-01 10:06:00#2` It will be processed since this is the first record.

Output:

```
-------------------------------------------
Batch: 1
-------------------------------------------
+------------------------------------------+---+
|window                                    |sum|
+------------------------------------------+---+
|[2021-01-01 10:05:00, 2021-01-01 10:10:00]|2  |
+------------------------------------------+---+
```

Note: `Batch:0` was empty.

Since size of the window is `5 minutes` we have a window value of `[2021-01-01 10:05:00, 2021-01-01 10:10:00]`

2. **Second record :** `2021-01-01 10:13:00#5`

We have only processed one record so far, which means `max event time = 2021-01-01 10:06:00` . Hence, the watermark (`max event time – delayThreshold`) is `2021-01-01 09:56:00` . For a window of size `5 minutes` , the watermark window will be `[09:55:00, 10:00:00]` . So any data earlier than the start of the watermark window (`2021-01-01 09:55:00`) will be discarded. However, the event time (`2021-01-01 10:13:00`) for the second record is greater than the start time of the watermark window, so it will be processed.

Note: in the watermark window, I only included time because date will always be `2021-01-01` in our dataset.

```
-------------------------------------------
Batch: 2
-------------------------------------------
+------------------------------------------+---+
|window                                    |sum|
+------------------------------------------+---+
|[2021-01-01 10:10:00, 2021-01-01 10:15:00]|5  |
+------------------------------------------+---+
```

3. **Third Record:** `2020-12-31 10:06:00#1`

Our system's current state is

- max event time: `[2021-01-01 10:13:00]`
- watermark: `[2021-01-01 10:03:00]`

- watermark window: `[2021-01-01 10:00:00, 2021-01-01 10:05:00]`

For this event, the timestamp is less than the start of the watermark window, hence it will be treated as late data and discarded. We will get an empty batch.

```
-------------------------------------------
Batch: 4
-------------------------------------------
+------+---+
|window|sum|
+------+---+
+------+---+
```

4. **Fourth Record:** `2021-01-01 10:08:00#8`

- max event time: `[2021-01-01 10:13:00]`
- watermark: `[2021-01-01 10:03:00]`
- watermark window: `[10:00:00, 10:05:00]`

This event's time is greater than the start of the watermark window, hence it will be processed.

```
-------------------------------------------
Batch: 5
-------------------------------------------
+-----------------------------------------+---+
|window                                   |sum|
+-----------------------------------------+---+
|[2021-01-01 10:05:00, 2021-01-01 10:10:00]|10 |
+-----------------------------------------+---+
```

5. **Fifth Record:** `2021-01-01 10:00:01#7`

- max event time: `[2021-01-01 10:13:00]`
- watermark: `[2021-01-01 10:03:00]`
- watermark window: `[10:00:00, 10:05:00]`

This event's time is slightly greater than the start of the watermark window, hence it will be processed.

6. **Sixth Record:** `2021-01-01 9:59:00#3`

- max event time: `[2021-01-01 10:13:00]`
- watermark: `[2021-01-01 10:03:00]`
- watermark window: `[10:00:00, 10:05:00]`

This event's time is less than the start of the watermark window, hence it will not be processed.

Here is the consolidated list of all ingested events with max event seen so far, watermark, watermark window, and if they were processed or not.

Data	Max event time so far	Watermark	Watermark window	Processed
2021-01-01 10:06:00#2				Y
2021-01-01 10:13:00#5	[2021-01-01 10:06:00]	[2021-01-01 09:56:00]	[09:55:00, 10:00:00]	Y
2020-12-31 10:06:00#1	[2021-01-01 10:13:00]	[2021-01-01 10:03:00]	[10:0:00, 10:05:00]	N
2021-01-01 10:08:00#8	[2021-01-01 10:13:00]	[2021-01-01 10:03:00]	[10:0:00, 10:05:00]	Y
2021-01-01 10:00:01#7	[2021-01-01 10:13:00]	[2021-01-01 10:03:00]	[10:0:00, 10:05:00]	Y
2021-01-01 9:59:00#3	[2021-01-01 10:13:00]	[2021-01-01 10:03:00]	[10:0:00, 10:05:00]	N
2021-01-01 10:25:10#4	[2021-01-01 10:13:00]	[2021-01-01 10:03:00]	[10:0:00, 10:05:00]	Y
2021-01-01 10:14:00#10	[2021-01-01 10:25:10]	[2021-01-01 10:15:10]	[10:15:00, 10:25:10]	N
2021-01-01 10:39:00#15	[2021-01-01 10:25:10]	[2021-01-01 10:15:10]	[10:15:00, 10:25:10]	Y
2021-01-01 10:26:00#1	[2021-01-01 10:39:00]	[2021-01-01 10:29:00]	[10:25:00, 10:30:00]	Y
2021-01-01 10:24:00#6	[2021-01-01 10:39:00]	[2021-01-01 10:29:00]	[10:25:00, 10:30:00]	N
2021-01-01 10:30:00#11	[2021-01-01 10:39:00]	[2021-01-01 10:29:00]	[10:25:00, 10:30:00]	Y

An image that demonstrates watermarking with an example.

Ingest above data into our streaming application and verify the processed status. You can find the complete code on ***GitHub***.

> I hope you have enjoyed learning about watermarking in Spark Streaming!



Reference

- https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html#handling-late-data-and-watermarking

Programming | Software Engineering | Big Data | Spark | Streaming



Published in Expedia Group Technology

5K Followers · Last published Feb 25, 2025

Follow

Stories from the Expedia Group Technology teams



Written by Neeraj Bhadani

696 Followers · 25 Following

Follow

Data Science and cloud computing enthusiast

No responses yet

A Akhedkar

What are your thoughts?

More from Neeraj Bhadani and Expedia Group Technology





In Expedia Group Technology by Neeraj Bhadani

Deep dive into Apache Spark Window Functions

Window functions operate on groups of data and return values for each record or group

Jun 25, 2020 557 4

In Expedia Group Technolo... by Christine Mathies...

A Short Introduction to Apache Iceberg

The data lake team at Expedia Group starts working with table formats, adds Hive...

Jan 26, 2021 662 3



In Expedia Group Tech... by Avinash Reddy Penu...

How to Import Public Certificates into Java's Truststore from a...

Troubleshooting troublesome certificates

Jul 2, 2020 238



In Expedia Group Technology by Neeraj Bhadani

Deep Dive into Apache Spark Array Functions

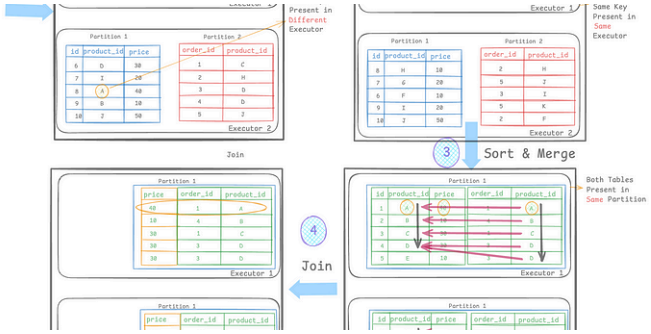
A practical guide to using array functions

Jul 16, 2020 201 3

See all from Neeraj Bhadani

See all from Expedia Group Technology

Recommended from Medium

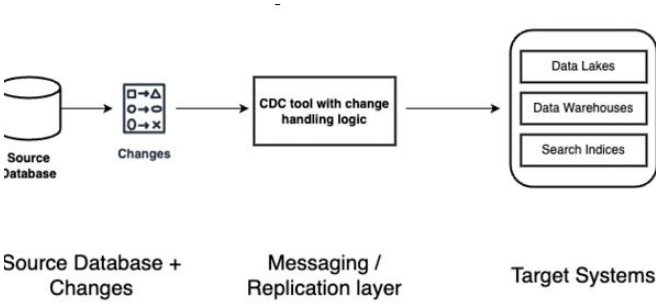


Shantanu Tripathi

Sort Merge Join: Visual Flow inside Executors

Learn the default join strategy used by Spark through visuals.

Dec 29, 2024 60



Hung Vo

Approaches to Change Data Capture (CDC) in Data Lakes:...

Learn to implement Change Data Capture (CDC) in data lakes using Apache Hudi & AW...

Jan 5 1



In Data Engineer Things by Ankush Sharma

The Hidden Optimization Behind Spark's mapPartitions

Iterator-to-Iterator Transformations

Nov 2, 2024 76

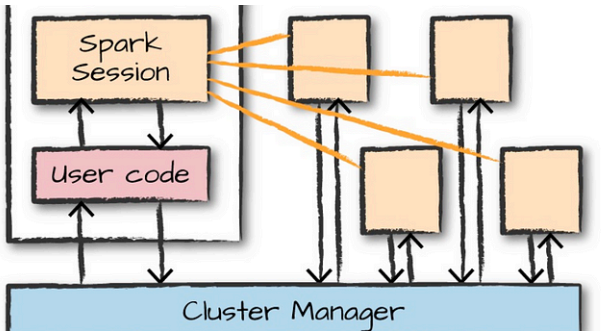


Kiran vutukuri

Working with Multiple Streams in Apache Spark(Part-2)

While working with a single stream is common, many real-world scenarios require...

Nov 29, 2024



In Towards Dev by Prem Vishnoi(cloudvala)

Apache Spark Architecture :A Deep Dive into Big Data Processing

Agenda

Feb 6 90 1



Ankita Patel

Building Data Pipelines with Apache Spark and Kafka

In today's digital world, data is the lifeblood of modern businesses. Organizations are...

Oct 15, 2024 51

See more recommendations